

Projekt POP — Dokumentacja końcowa

Filip Fijałkowski, Joanna Jósko, Jakub Sikorski

Krótkie streszczenie założeń z Projektu Wstępnego

W dokumentacji wstępnej przyjęliśmy, że dla algorytmu ewolucyjnego:

- szyfrowana podsieć będzie heurystycznie wyznaczana jako niezmiennie MST dla całej populacji;
- ograniczenia stanu rozwiązań będą miękkie;
- zastosujemy sukcesję elitarną, selekcję turniejową, krzyżowanie równomierne oraz mutację rozkładem Poissona.

Zaplanowane zostały eksperymenty na 2 różnych topologiach (*full-mesh* i *rzeczywiste przykłady z SNDlib*) oraz 3 różnych rozmiarach sieci.

Pełen opis funkcjonalny

Celem projektu jest porównanie optymalizacji przy użyciu modelu MIP (*Mixed Integer Programming*) oraz algorytmu ewolucyjnego. Projekt składa się z następujących elementów:

- napisanej przez nas biblioteki `net_opt`, a w niej:
 - moduł `core` z poszczególnymi elementami algorytmu ewolucyjnego (*ograniczenia, mutacja, krzyżowanie, selekcja, warunki stopu, reprezentacja populacji*) i klasą agregującą wszystko, by uruchomić algorytm;
 - moduł `utils` zawierający niektóre metody pomocnicze, m.in. generowanie plików `.dat` na podstawie przekazanych zmiennych, “optymalne” rozkłady stanów początkowych algorytmu, parser dla plików z bazy SNDlib, wizualizacje osobników;
- katalogu `tests` z testami jednostkowymi elementów biblioteki `net_opt`;
- katalogu `AMPL` z modelami MIP i przykładowymi danymi;
- katalogu `SNDlib` z przykładami sieci rzeczywistych;
- katalogu `experiment_data_logs` z plikami JSON zawierającymi dane z wykonanych masowo eksperymentów;
- Jupyter Notebook `NetworkEvolution` z komórkami kodu do wykonania eksperymentów i porównania wyników;
- Jupyter Notebook `Experiments` z komórkami kodu do wykonania zagregowanych eksperymentów badających wpływ hiperparametrów algorytmu ewolucyjnego oraz ładowania wyników do plików JSON.

Precyzyjny opis algorytmów oraz opis zbiorów danych

Szczegółowy opis problemu

Problem polega na znalezieniu optymalnego (tj. z *minimalnym kosztem transponderów*) schematu połączeń między węzłami szyfrowanej sieci optycznej.

Częścią rozwiązania jest też wyznaczenie podsieci szyfrowanych krawędzi, łączącej wszystkie węzły. Każdy transponder zajmuje pewną długość fali na krawędzi, szyfrowane krawędzie mają węższe pasmo od zwykłych.

Dla każdej wartości, jeśli nie jest to inaczej doprecyzowane, domyślnie przyjmujemy zbiór wartości nieujemnych liczb całkowitych.

Parametry

- Struktura sieci (*spójny graf nieskierowany*):
 - N oznacza zbiór węzłów;
 - E oznacza zbiór krawędzi interpretowanych bezkierunkowo;
 - F oznacza zbiór krawędzi interpretowanych kierunkowo¹;
 - $\{i, j\} \in E$ oznacza krawędź "fizyczną" między węzłami i oraz j ;
 - $(i, j) \in F$ oznacza krawędź między węzłami i oraz j w kontekście abstrakcyjnego, kierunkowego "przepływu";
 - a oznacza szerokość pasma nieszyfrowanego (*maksymalna liczba połączeń przechodzących przez krawędź*);
 - b oznacza szerokość pasma szyfrowanego.
- Pojemność i koszt rodzajów transponderów obsługujących połączenia między węzłami:
 - T oznacza zbiór rodzajów transponderów;
 - $m \in T$ oznacza rodzaj transpondera;
 - c_m oznacza pojemność (*liczba rzeczywista*);
 - p_m oznacza koszt (*liczba rzeczywista*).
- Zapotrzebowanie na przepustowość połączeń między węzłami:
 - C oznacza zbiór wszystkich par węzłów (*połączeń*);
 - $\{k, l\} \in C$ oznacza połączenie między parą węzłów k oraz l
 - d_{kl} oznacza zapotrzebowanie na przepustowość² połączenia $\{k, l\} \in C$.

¹ Choć zakładamy, że sam graf jest nieskierowany, kierunkowa interpretacja krawędzi była konieczna, aby zrealizować ograniczenia związane ze spójnością wyznaczanych ścieżek połączeń.

² Zarówno przygotowany zbiór modeli MIP, jak i algorytm ewolucyjny pozwalają na dowolne określenie zapotrzebowań przepustowości, ale w praktyce przyjęliśmy testowanie z identyczną wartością dla każdej pary węzłów.

Zmienne

- Szyfrowanie krawędzi:
 - binarne e_{ij} oznacza stan zaszyfrowania krawędzi ($1 \Leftrightarrow$ szyfrowana).
- Zajętość pasm krawędzi:
 - u_{ijkl} oznacza liczbę ścieżek połączeń $\{k, l\} \in C$ przechodzących przez daną krawędź $(i, j) \in F$.
 - pomocnicze oznaczenie s_{kl} oznacza łączną liczbę ścieżek połączeń $\{k, l\} \in C$ wychodzących z węzła "startowego" k :

$$s_{kl} = \sum_{j \in N \mid (k,j) \in F} u_{kjkl}$$

- Wybór transponderów:
 - t_{klm} oznacza liczbę transponderów typu $m \in T$ obsługujących połączenia $\{k, l\} \in C$.

Funkcja celu

Minimalizujemy łączny koszt transponderów, określany wzorem: $\sum_{\{k,l\} \in C, m \in T} (p_m t_{klm})$.

Ograniczenia

Spełnienie zapotrzebowania

$$\forall \{k, l\} \in C \quad \sum_{m \in T} (u_{klm} c_m) \geq d_{klm}$$

Dla każdej pary węzłów $\{k, l\} \in C$, suma pojemności obsługujących je transponderów musi być nie mniejsza niż zapotrzebowanie na przepustowość połączenia.

Zgodność liczby transponderów z liczbą połączeń

$$\forall \{k, l\} \in C \quad \sum_{m \in T} u_{klm} = s_{kl}$$

Dla każdej pary węzłów $\{k, l\} \in C$, łączna liczba obsługujących je transponderów musi być równa łącznej liczbie wychodzących z k ścieżek połączeń między tą parą węzłów.

Spójność ścieżek połączeń

$$\forall \{k, l\} \in C \quad \forall i \in N \setminus \{k, l\} \quad \sum_{j \in N \mid (i,j) \in F} u_{ijkl} = \sum_{j \in N \mid (j,i) \in F} u_{jikl}$$

Dla każdej pary węzłów $\{k, l\} \in C$ i każdego wierzchołka i nienależącego do tej pary, łączne liczby wchodzących do oraz wychodzących z i ścieżek połączeń między tą parą węzłów muszą być równe.

Poprawne zakończenie połączeń

$$\forall \{k, l\} \in C \quad \sum_{i \in N \mid (i,l) \in F} u_{ilkl} = s_{kl}$$

Dla każdej pary węzłów $\{k, l\} \in C$, łączna liczba ścieżek połączeń między tą parą węzłów wchodzących do l musi być równa łącznej liczbie ścieżek wychodzących z k .

Maksymalna szerokość pasm krawędzi

$$\forall \{i, j\} \in E \quad \sum_{\{k,l\} \in C} (u_{ijkl} + u_{jikl}) \leq a + e_{ij}(b - a)$$

Dla każdej krawędzi $\{i, j\} \in E$, łączna liczba przechodzących przez tę krawędź połączeń musi być nie większa niż szerokość pasma tej krawędzi (zgodnie z jej stanem szyfrowania).

Model MIP

Oprócz powyższych, model MIP używa dodatkowego parametru, zmiennej i ograniczeń w celu wyznaczenia podsieci szyfrowanej.

Parametr — “korzeń” drzewa szyfrowania

$r \in N$ oznacza (arbitralnie wybrany) węzeł, dla którego sprawdzamy, czy istnieje szyfrowana ścieżka do każdego innego węzła.

Zmienna — “przepływ” szyfrowania

Binarne f_{ijk} oznacza, czy przez krawędź $(i, j) \in F$ przechodzi ścieżka szyfrowania między r a $k \in N \setminus \{r\}$.

Ograniczenia

Warunki istnienia szyfrowanej podsieci

$$\forall k \in N \setminus \{r\} \quad \sum_{j \in N \mid (r,j) \in F} f_{rjk} = 1 + \sum_{i \in N \mid (i,r) \in F} f_{irk}$$

Dla każdego wierzchołka $k \in N$ niebędącego korzeniem, łączna liczba szyfrowanych połączeń między korzeniem a k wychodzących z korzenia musi być większa o 1 od liczby połączeń wchodzących do korzenia.

$$\forall i \in N \setminus \{r\} \quad \forall k \in N \setminus \{r, i\} \quad \sum_{j \in N \mid (i,j) \in F} f_{ijk} = \sum_{j \in N \mid (j,i) \in F} f_{jik}$$

Dla każdych różnych od siebie wierzchołków $i, k \in N$ niebędących korzeniem, łączna liczba szyfrowanych połączeń między korzeniem a k wychodzących z i musi być równa liczbie połączeń wchodzących do i .

Uzasadnienie

Powyższe warunki zapewniają, że istnieją ścieżki szyfrowane między korzeniem a każdym z pozostałych węzłów, co implikuje, że istnieją ścieżki szyfrowane między każdą parą węzłów (*każda ścieżka $i \rightarrow j$ będzie konkatenacją ścieżek $i \rightarrow r$ oraz $r \rightarrow j$*).

Wynika to z tego, że jeśli graf jest spójny, a w każdym wierzchołku ścieżki poza końcowym k jest zachowana równość In/Out (w przypadku korzenia zakładamy 1 dodatkowe połączenie wchodzące "spoza" sieci, żeby uniemożliwić zakończenie ścieżki w nim), to ścieżka musi się zakończyć w wierzchołku k .

Zgodność stanu szyfrowania krawędzi z wyznaczonymi ścieżkami szyfrowania

$$\forall \{i, j\} \in E \quad \forall k \in N \setminus \{r\} \quad f_{ijk} + f_{jik} \leq 2e_{ij}$$

Dla każdej krawędzi $\{i, j\} \in E$ i każdego wierzchołka $k \in N$ niebędącego korzeniem, przez krawędź $\{i, j\}$ może przepływać połączenie szyfrowane do k tylko jeśli krawędź jest oznaczona jako szyfrowana.

Algorytm ewolucyjny

Algorytm ewolucyjny odrobinę odbiega od zdefiniowanych w opisie problemu ograniczeń, m.in. poprzez luźniejsze ich przestrzeganie oraz zachłanne wyznaczenie podsieci szyfrowanej.

Ograniczenia

Algorytm ewolucyjny ma zaimplementowane łagodniejsze wersje niektórych ograniczeń (*odpowiednie nierówności zamiast równości*) w celu umożliwienia eksploracji (*aby np. pojawiła się nowa ścieżka, zachowując spełnione ograniczenie dot. spójności ścieżek, konieczna byłaby jednoczesna mutacja na każdej krawędzi ścieżki, co jest bardzo mało prawdopodobne*).

Stopień przekroczenia ograniczenia

Każde ograniczenie ma przypisaną funkcję zwracającą stopień jego przekroczenia w zakresie $[0, 1]$, gdzie 0 oznacza całkowite spełnienie wymagań danego ograniczenia.

Odbywa się to na zasadzie porównania wartości a i b , między którymi dane ograniczenie wymaga odpowiedniej relacji, i obliczenia dla nich średniej wartości “macierzy podobieństwa”.

W przypadku ograniczeń wymagających równości wartości każdy element macierzy podobieństwa jest równy $\frac{a-b}{\max\{a,b\}}$.

W przypadku “miękkich” wariantów tych ograniczeń, każdy element macierzy podobieństwa jest równy $\frac{\text{ReLU}(a-b)}{\max\{a,b\}}$, zakładając, że oczekiwane jest $b \geq a$.

Ograniczenia, które z samej definicji problemu były nierównościami, są traktowane wyjątkowo — zamiast średniej z macierzy podobieństwa obliczana jest proporcja odpowiednich sum. Takie rozwiązanie zapewnia znacznie wyższy stopień przekroczenia i twardsze ich przestrzeganie (*przyjeliśmy, że zapotrzebowanie i szerokość pasm są kluczowymi do spełnienia ograniczeniami*).

Wzór:
$$\frac{\sum_i \text{ReLU}(a_i - b_i)}{\sum_i \max\{a_i, b_i\}}$$

Wyznaczanie podsieci szyfrowanej

Aby usprawnić działanie algorytmu poprzez redukcję wymiarowości przestrzeni przeszukiwań, postanowiliśmy rozbić zadanie na 2 podproblemy: heurystyczne wyznaczenie podsieci szyfrowanej oraz ewolucyjną optymalizację schematu połączeń i transponderów.

Przed główną pętlą algorytmu ewolucyjnego wyznaczana jest niezmienna oraz identyczna dla każdego osobnika podsieć szyfrowana jako MST (*Minimum Spanning Tree*) zwracane przez (*lekko zmodyfikowany*) algorytm Kruskala.

Metryką stosowaną jako waga krawędzi jest miara pośrednictwa krawędzi (*ang. “edge betweenness centrality” — oznacza to liczbę najkrótszych ścieżek w grafie przechodzących przez daną krawędź*). Modyfikacja algorytmu polega głównie na tym, że w przypadku “remisu” dodatkowym, decydującym kluczem sortowania jest obliczana dynamicznie, dla każdej iteracji, miara pośrednictwa traktująca graf, jakby nie zawierał krawędzi już należących do wyznaczanego MST.

Ten sposób sortowania krawędzi wynika z chęci uniknięcia sytuacji, w której algorytm np. dla sieci full-mesh zwraca jako “optymalną” sieć-gwiazdę dookoła pierwszego wierzchołka. Podczas opracowywania algorytmu doszliśmy do wniosku, że taki schemat sieci znacząco ograniczałby możliwość nawiązywania połączeń między tym wierzchołkiem a innymi.

Należy również zaznaczyć, że zastosowana metryka zakłada równe zapotrzebowanie między każdą parą węzłów (*ponieważ nie bierze pod uwagę wartości zapotrzebowania, tylko samą liczbę ścieżek*), dlatego eksperymenty

porównujące działanie algorytmu z modelem MIP wykonaliśmy na tak ustalonym zapotrzebowaniu.

Wykonanie algorytmu ewolucyjnego

Ze względu na uprzednie wyznaczenie stałej podsieci szyfrowanej, genotyp osobników jest w całości reprezentowany nieujemnymi liczbami całkowitymi odpowiadającymi zajętości pasm krawędzi i wyborze transponderów.

Każdy osobnik reprezentuje dane rozwiązanie problemu *(ze względu na miękkość ograniczeń, niekoniecznie całkowicie zgodne ze wszystkimi wymaganiami)*.

Miara jakości

Początkowa wersja algorytmu zakładała dodawanie do funkcji kosztu sumy ważonej kar za niedotrzymanie opisanych ograniczeń. Doprowadzało to do sytuacji, w której np. *(przy równych wagach ograniczeń)* spełnienie całkowicie jednego wymagania kosztem zupełnego zignorowania pozostałych było oceniane identycznie jak spełnienie równo każdego wymagania. Innym problemem była sytuacja patologiczna, w której algorytm mógł potraktować np. osobnika z brakiem transponderów za pozornie najlepszego, jeśli funkcja kary zwracała mniejszy wynik niż optymalna funkcja kosztu.

Aby usprawnić działanie algorytmu i ułatwić dobór hiperparametrów, zdecydowaliśmy się na zastosowanie następującego wzoru:

$q(x) = \frac{f(x)}{p(x)} + w(1 - p(x))$, gdzie:

- x oznacza danego osobnika;
- $q(x)$ oznacza miarę jakości osobnika;
- $f(x)$ oznacza minimalizowaną funkcję celu;
- w oznacza stałą wartość kary;
- $p(x)$ oznacza funkcję kary równą $\prod_{o \in O} (1 - o(x))$, gdzie:
 - O oznacza zbiór funkcji zwracających stopień przekroczenia danego ograniczenia;
 - $o \in O$ oznacza daną funkcję stopnia przekroczenia ograniczenia *(zbiór wartości $[0, 1]$, gdzie zero oznacza spełnienie ograniczenia)*.

Połączenie wykładniczego skalowania z każdym kolejnym przekroczonym ograniczeniem oraz uzyskiwania wartości ∞ w przypadku całkowitego przekroczenia dowolnego ograniczenia skutkuje tym, że algorytm dąży do równomiernego spełniania ograniczeń *(spełnienie jednego ograniczenia zazwyczaj nie ma sensu, jeśli pozostałe nie są spełnione, a szczególnie jeśli jedno jest całkowicie przekroczone)*.

Selekcja

Algorytm wykorzystuje selekcję turniejową z rozmiarem równym 3. Przyjeliśmy, że zapewnia ona większą zdolność eksploatacji, zachowując równocześnie miękkość selekcji.

Krzyżowanie

Wszystkie geny osobników są liczbami całkowitymi, dlatego wybraliśmy krzyżowanie równomierne, aby uniknąć niepotrzebnych operacji zaokrąglania wartości.

Algorytm został zaimplementowany w taki sposób, aby umożliwić niezależną modyfikację parametru CR dla genów odpowiedzialnych za zajętość krawędzi i genów odpowiedzialnych za wybór transponderów.

Wzór krzyżowania (*aplikowany niezależnie dla każdego genu*):

$$c_1 := rp_1 + (1 - r)p_2, \quad c_2 := rp_2 + (1 - r)p_1, \text{ gdzie:}$$

- c_1, c_2 oznaczają geny potomstwa;
- p_1, p_2 oznaczają geny rodziców;
- r to binarna zmienna losowa z rozkładu zero-jedynkowego o p równym wartości wyznaczonego dla danej kategorii genu parametru CR.

Mutacja

Ze względu na całkowitoliczbowość genów i chęć unikania zaokrąglania, zastosowaliśmy mutację polegającą na przypisaniu do genu zmiennej losowej z rozkładu Poissona o średniej równej aktualnej wartości genu.

W przypadku mutacji zajętości krawędzi zachodzi drobna naprawa lamarkowska — wylosowana wartość jest rzutowana tak, aby nie przekroczyć szerokości pasma danej krawędzi.

Wzory mutacji (*aplikowane niezależnie dla każdego genu*):

$$u := (1 - r)u + r \cdot \min\{\text{Pois}(u), e\}, \quad t := (1 - r)t + r \cdot \text{Pois}(t), \text{ gdzie:}$$

- u, t oznaczają odpowiednio geny zajętości krawędzi i wyboru transponderów;
- r to binarna zmienna losowa z rozkładu zero-jedynkowego o p równym odpowiedniej szansie na mutację dla danej kategorii genu;
- $\text{Pois}(\lambda)$ oznacza zmienną losową z rozkładu Poissona dla danej λ ;
- e oznacza szerokość pasma krawędzi, której dotyczy dany gen u (*bierze pod uwagę stan szyfrowania*).

Sukcesja

Ostatecznie zdecydowaliśmy się utrzymywać elitę najlepszych n_e osobników zamiast rozważanej w dokumentacji wstępnej elity osobników spełniających wszystkie ograniczenia, ponieważ algorytm w zasadzie bardzo rzadko osiągał takie idealne stany.

Zbiór danych

Sieci topologii full-mesh są generowane przy użyciu podanej liczby węzłów (w odpowiednim modelu AMPL zbiór węzłów jest definiowany jako $1..n$, zaś do algorytmu ewolucyjnego jest przekazywana trójkątna macierz jedynek jako macierz sąsiedztwa).

Przykłady sieci rzeczywistych są brane z bazy [SNDlib](#) jako pliki tekstowe ich natywnego formatu i odpowiednio parsowane do zmiennych w kodzie, aby wygenerować plik `.dat` dla AMPL i uzyskać argumenty dla algorytmu ewolucyjnego. Jako testowane rodzaje transponderów przyjęliśmy przykładowe wartości podane na konsultacjach z początku semestru:

- typ A: pojemność 40, koszt 1;
- typ B: pojemność 100, koszt 3;
- typ C: pojemność 200, koszt 5;
- typ D: pojemność 400, koszt 9;
- typ E: pojemność 800, koszt 11.

Parametr zapotrzebowania, szczególnie dla sieci rzeczywistych, staraliśmy się dobrać do momentu, w którym AMPL zwracał błąd niewykonalnego zadania.

Raport z przeprowadzonych testów oraz wnioski

Przy wszystkich testach algorytmu ewolucyjnego wykorzystywane są wyniki uśrednione z 15 uruchomień algorytmu.

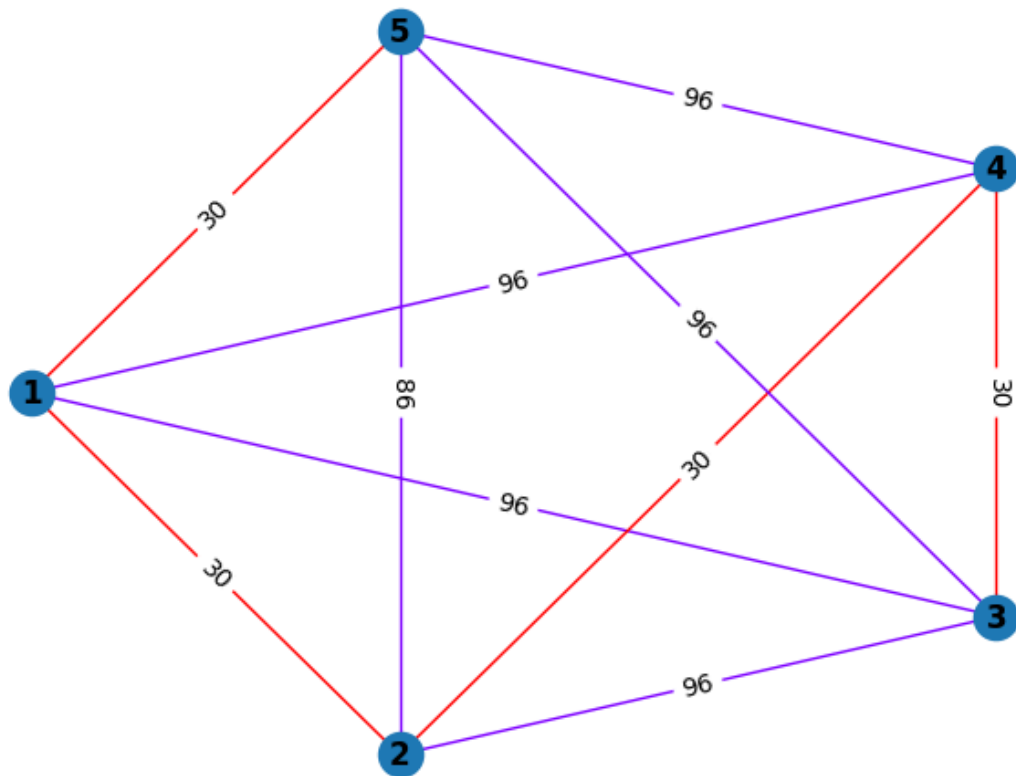
Do wykonania tak zagregowanych eksperymentów powstał wspomniany w opisie funkcjonalnym, oddzielny Notebook [Experiments](#).

Przy testowaniu sieci full-mesh 20 węzłów napotkaliśmy problemy techniczne spowodowane brakiem pamięci na maszynie używanej do testowania. Niestety nie znaleźliśmy przyczyny ani rozwiązania, potencjalnie problem mógł spowodować nienadający się garbage collector (*pojedyncze uruchomienie instancji algorytmu działa, lecz pętla 15 uruchomień w celu agregacji niedeterministycznych wyników rzuca błąd dot. braku pamięci*). Z tego powodu zabrakło statystyk dot. tych sieci.

Eksperymenty porównujące rozmiary populacji zostały oryginalnie wykonane dla stałej liczby iteracji, skutkiem czego wykresy porównujące łączne liczby ewaluacji okazały się mało czytelne (*krzywe zbieżności kończyły się w różnych miejscach*). Rozwiązaniem na to zostało obcięcie porównania ewaluacji do pierwszych 300 tysięcy ewaluacji (*odpowiadających wykonaniu algorytmu na najmniejszej badanej populacji przez wybraną wcześniej stałą liczbę iteracji*).

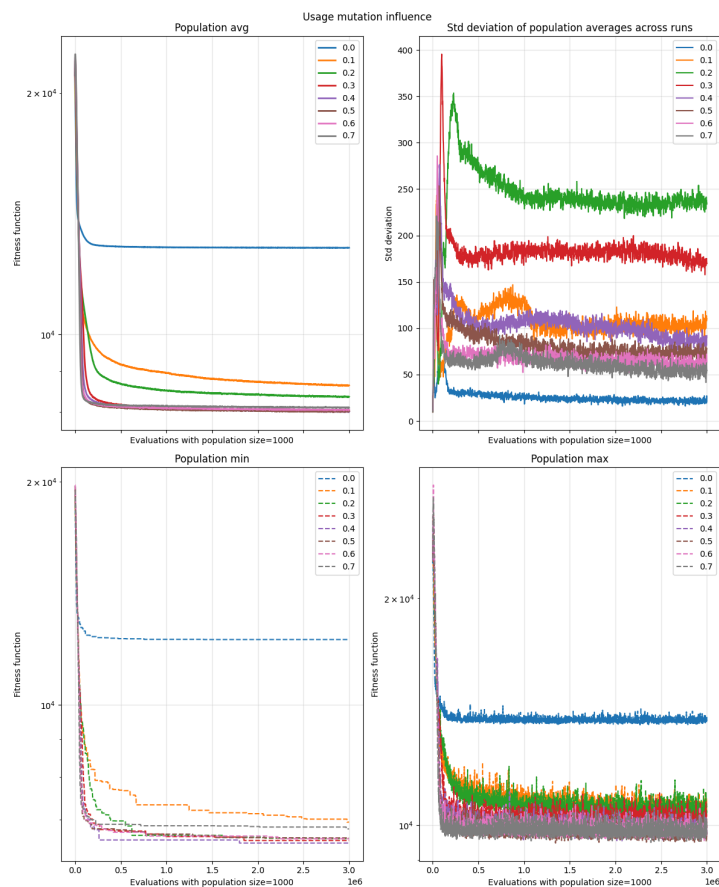
Sieć full mesh 5 węzłów

Wynik modelu MIP



Łączny koszt: 6050.

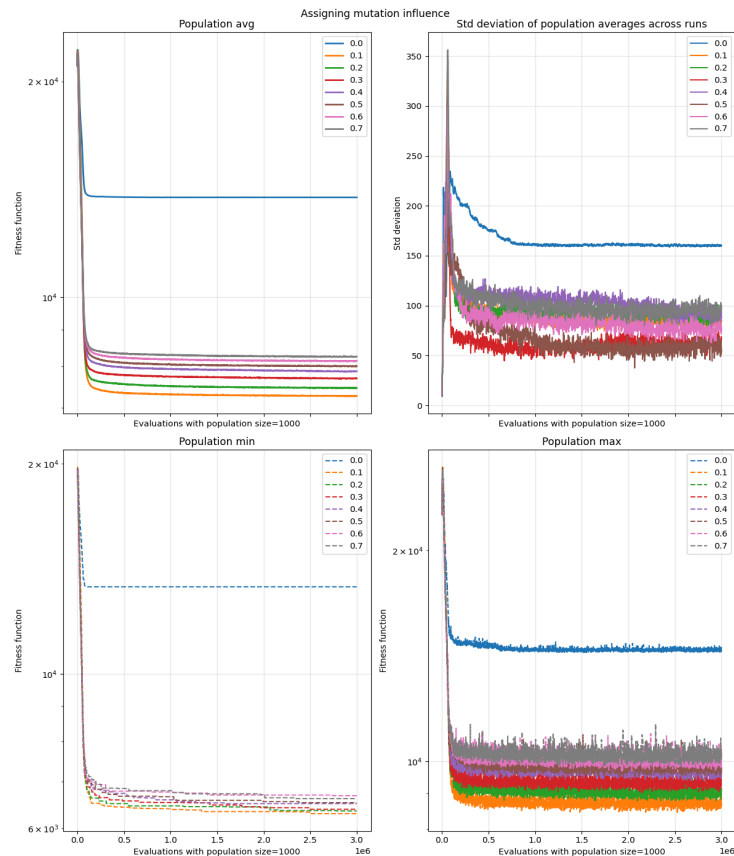
Mutacja zajętości krawędzi



Obserwacje i wnioski:

- Algorytm szybciej osiąga lepsze wyniki dla wysokich mutacji (0.3–0.7).
- Algorytm bardzo szybko zbiega do minimum lokalnego — potrzebuje zazwyczaj około 300 iteracji z populacją 1000 (łącznie 300 tys. ewaluacji).
- Najniższe odchylenie standardowe (pomijając wyłączenie mutacji) występuje dla wartości parametru 0.7.
- Dla wartości 0.7 zauważalny jest również długi okres braku zmian najlepszego (*min*) osobnika, co ostatecznie poskutkowało gorszą jakością od wartości 0.2–0.6.

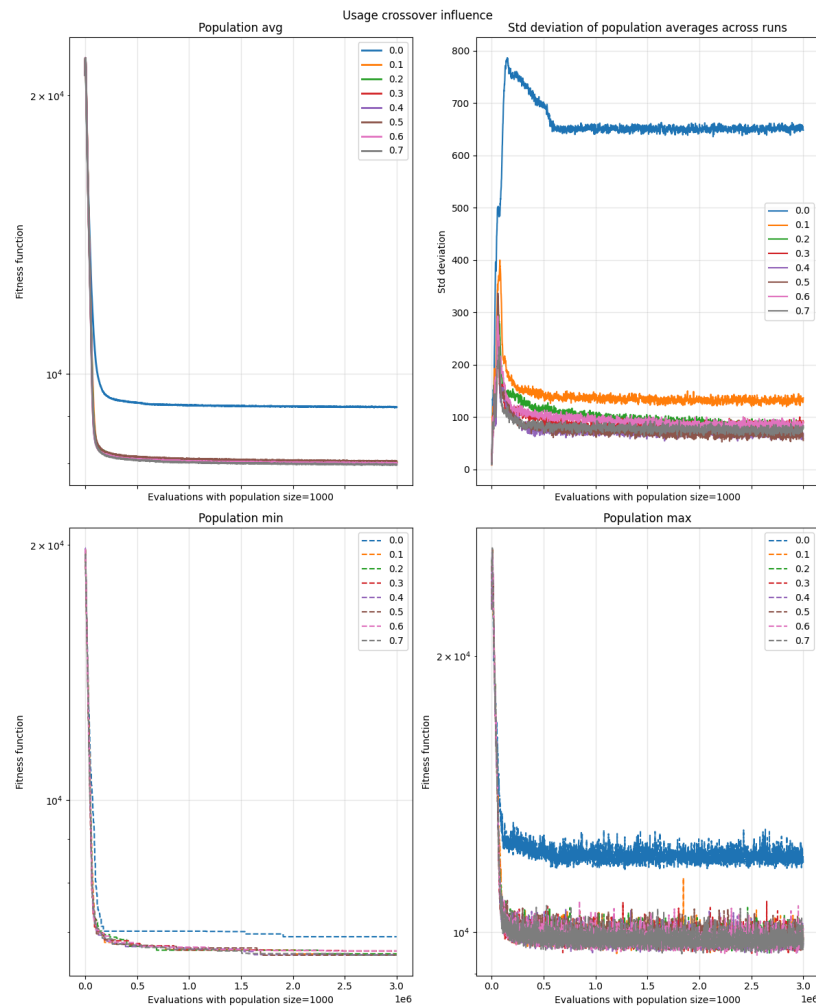
Mutacja wyboru transponderów



Obserwacje i wnioski:

- Wartość parametru 0.1 daje najlepsze wyniki, zbiega najszybciej i osiąga najniższą wartość średnią i minimalną.
- Odchylenia standardowe są w zasadzie podobne, choć można zauważyć najmniejsze wartości dla wartości parametru 0.3 i 0.7.
- W przeciwieństwie do mutacji zajętości, włączenie tej mutacji *zmniejsza* odchylenia standardowe. Może to wynikać ze stosowanej naprawy poprzez lamarkowskie rzutowanie.

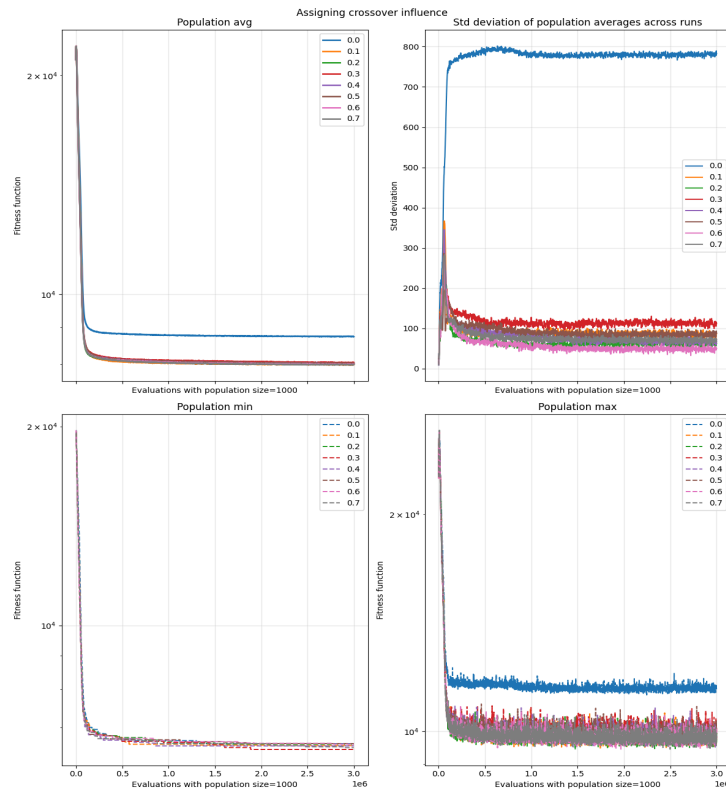
Krzyżowanie zajętości krawędzi:



Obserwacje i wnioski:

- Wszystkie niezerowe wartości dają praktycznie identyczne wyniki, z wyjątkiem 0.1, które uzyskało nieco wyższe odchylenia standardowe.
- Wartość parametru CR dla zajętości krawędzi nie wpływa znacząco na działanie algorytmu dla tak małych sieci — ważniejsze jest wyznaczenie dobrej wartości szansy mutacji.

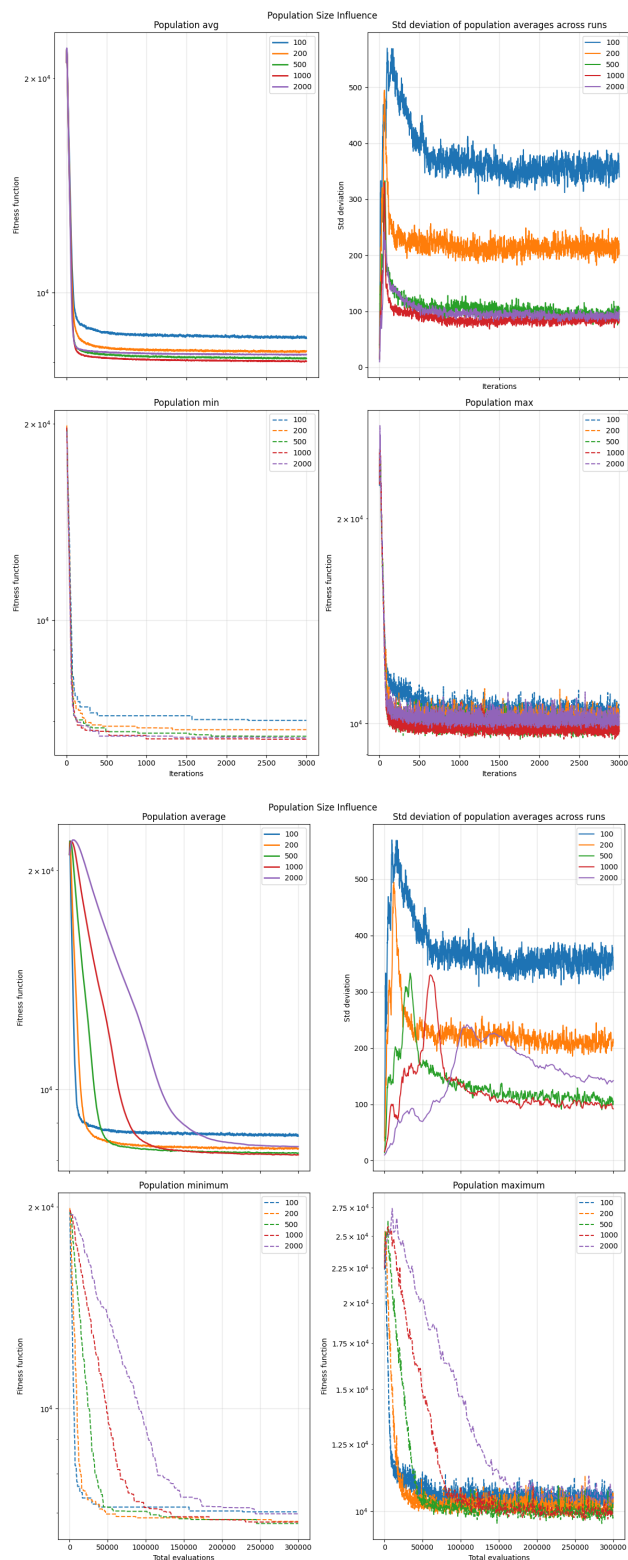
Krzyżowanie wyboru transponderów



Obserwacje i wnioski:

- Podobnie co w przypadku krzyżowania zajętości krawędzi — znaczenie ma tylko to, czy krzyżowanie jest w ogóle włączone (*tj. czy wartość parametru CR jest różna od 0 lub 1*).
- Dla wartości parametru 0.3 widać minimalnie lepsze wyniki najlepszych osobników, lecz również większe odchylenia standardowe w populacji.

Rozmiar populacji



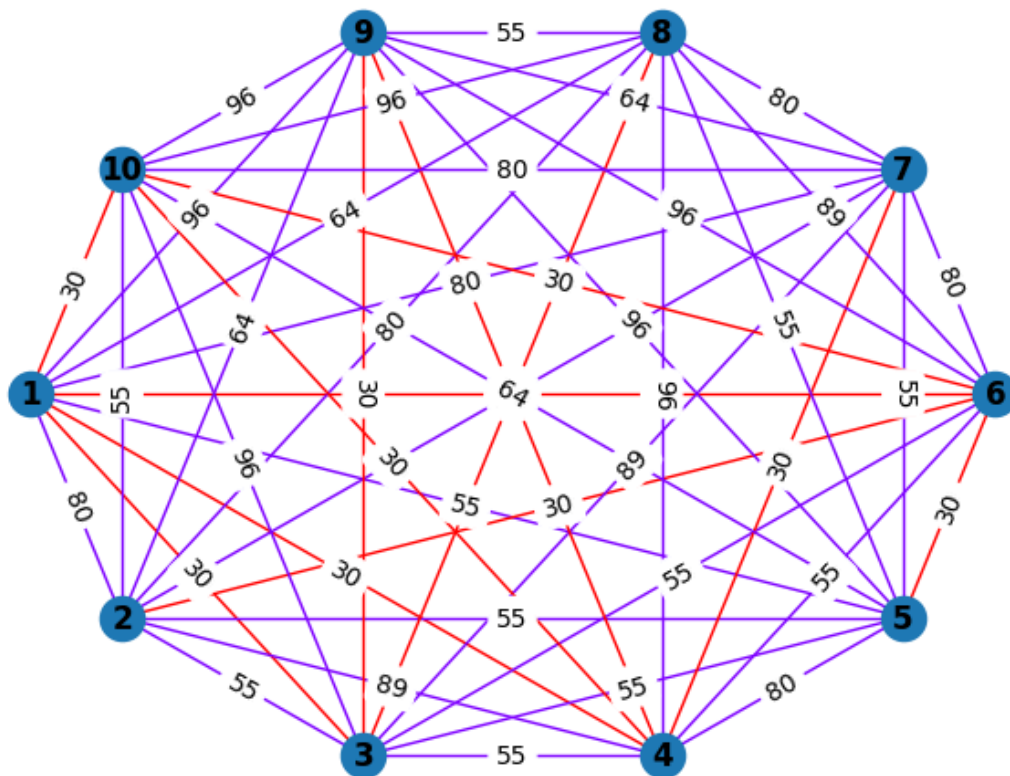
Obserwacje i wnioski:

- Patrząc na samą liczbę iteracji, optymalne wydaje się być 1000 osobników, lecz z wykresu porównującego łączne liczby ewaluacji (*iteracje wymnożone przez rozmiar populacji*) uzyskaliśmy inne wnioski.

- Porównując liczbę ewaluacji, najszybciej zbiega populacja 100 osobników, lecz najlepsze rozwiązanie w ciągu 300 tys. ewaluacji znalazła populacja 500 osobników.
- 1000 osobników dalej się wyróżnia najlepszym średnim wynikiem populacji.

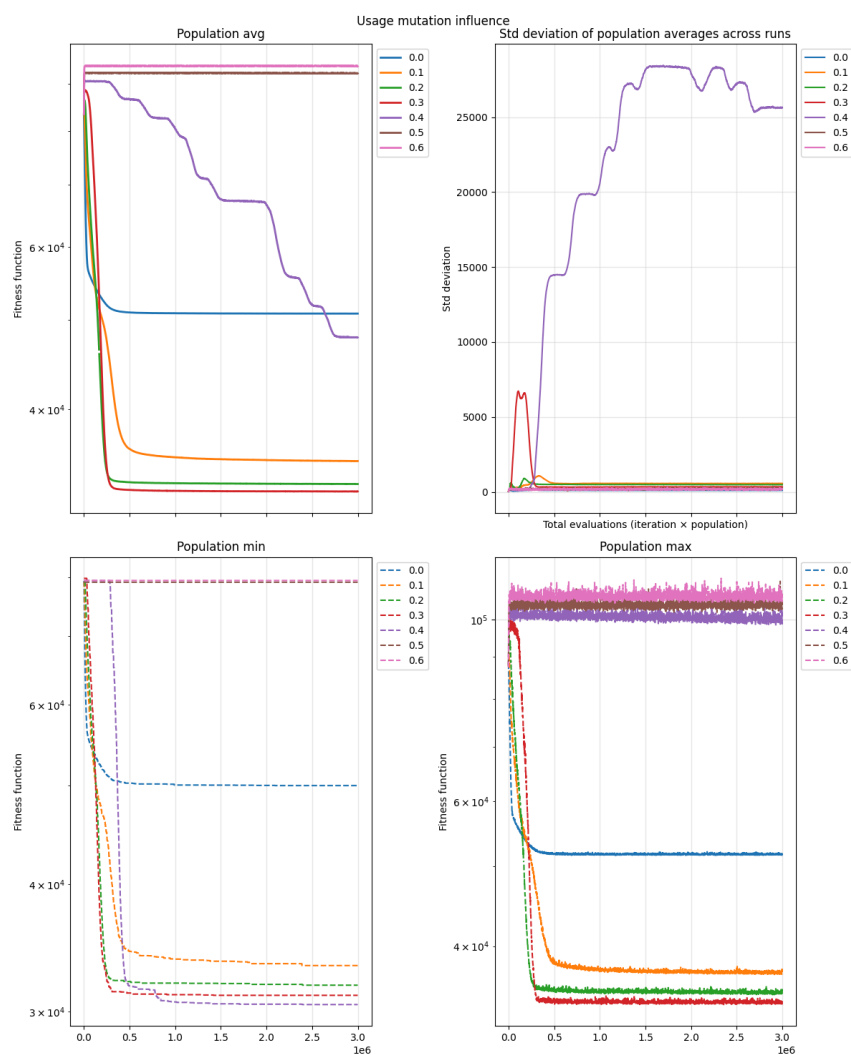
Sieć full mesh 10 węzłów

Wynik modelu MIP



Łączny koszt: 27225.

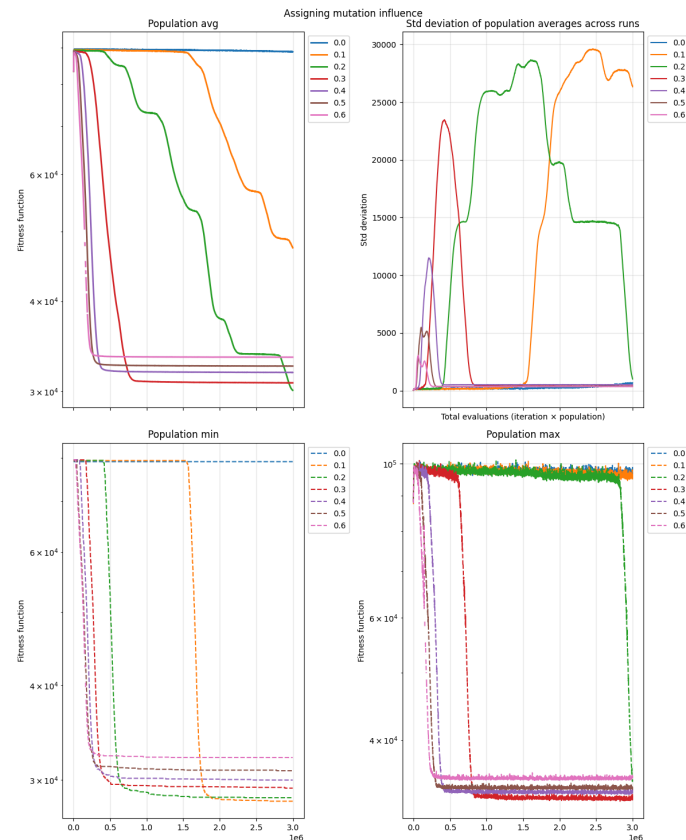
Mutacja zajętości krawędzi



Obserwacje i wnioski:

- Wartości 0.2–0.3 dają ogólnie najlepsze rezultaty, średnie wyniki ich populacji zbiegają najszybciej i najniżej.
- Wysokie wartości (0.5–0.6) zaskakująco uzyskały gorsze wyniki niż brak mutacji — jakość osobników pozostała bez zmian od stanu początkowego, co prawdopodobnie oznacza niezdolność do zachowania spełnionych ograniczeń.
- Najlepszy wynik pojedynczego osobnika osiągnął parametr 0.4, ale niektórym osobnikom jego populacji nie udało się jakkolwiek poprawić jakości, co skutkuje rosnącym odchyleniem standardowym.

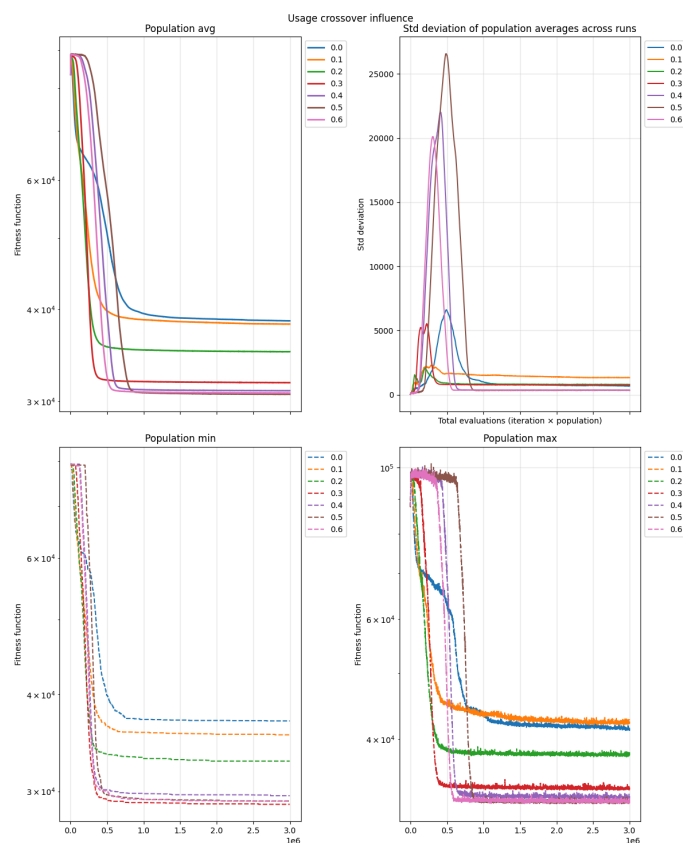
Mutacja wyboru transponderów



Obserwacje i wnioski:

- Wartości parametru 0.1–0.2 dają najniższe wyniki funkcji, ale jednocześnie mają bardzo wysokie maksymalne wartości.
- Im mniejsza wartość parametru, tym lepszy ostateczny wynik najlepszego osobnika, lecz również dłużej zajmuje jego osiągnięcie.
- Poprawa jakości najgorszego osobnika wymaga znacznie więcej iteracji niż osiągnięcie optimum lokalnego.

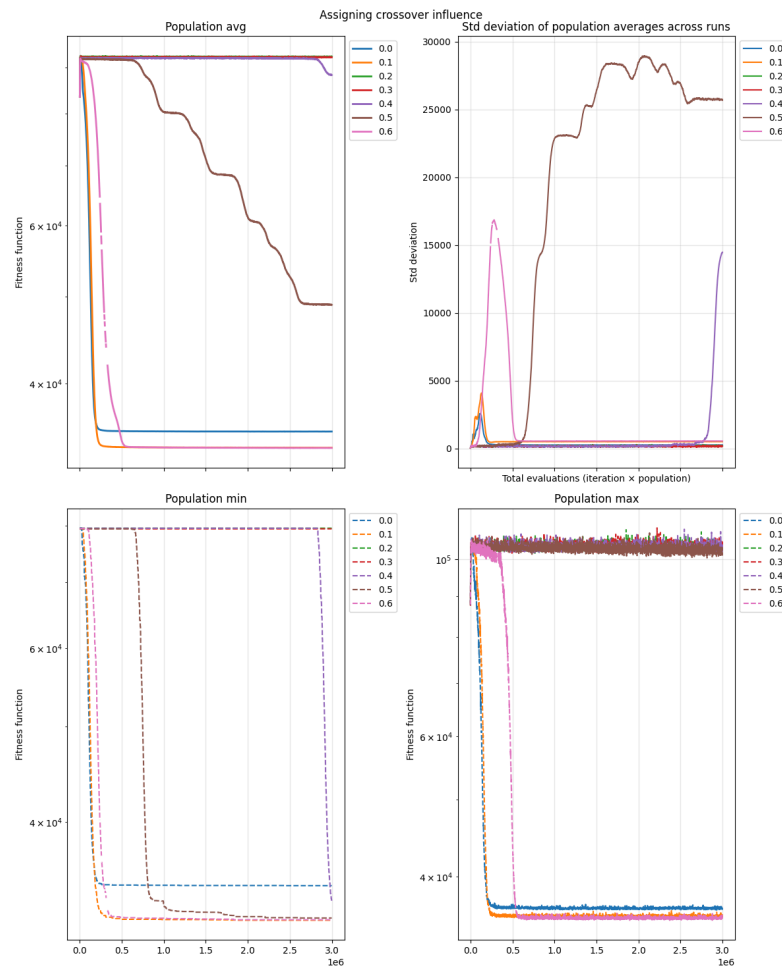
Krzyżowanie zajętości krawędzi



Obserwacje i wnioski:

- Wartości 0.3–0.6 dają podobne, ogólnie najlepsze wyniki.
- Wartość 0.3 uzyskała najlepszy wynik najlepszego osobnika, ale gorszą średnią populacji od wartości 0.4–0.6.
- Wartości 0.4–0.6 wyróżniają się również największymi szczytowymi wartościami odchylenia standardowego. Wynika to z późnej poprawy najgorszych osobników.
- Zgodnie z oczekiwaniami, wartości 0.4 i 0.6 mają bardzo podobne wykresy (ze względu na sposób działania krzyżowania są funkcjonalnie ekwiwalentne, gdyż kolejność rodziców nie ma znaczenia).

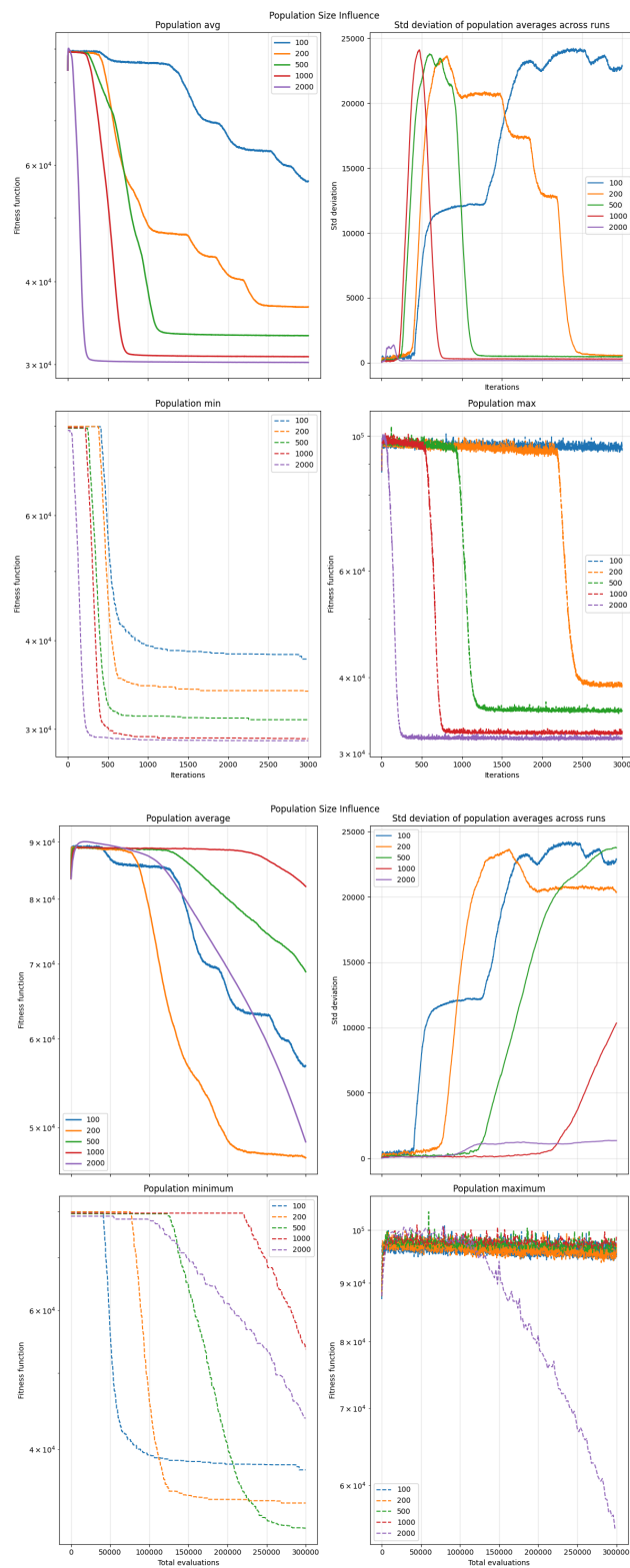
Krzyżowanie wyboru transponderów



Obserwacje i wnioski:

- 0.1 i 0.6 dają podobne wyniki, zarówno minimum/maximum jak i średnią, ale 0.1 szybciej zbiega i ma mniejsze odchylenie standardowe.
- 0.4 i 0.6 zaskakująco uzyskały skrajnie różne wyniki.
- Najgorsze wyniki uzyskały wartości 0.2–0.3, którym nie udało się poprawić stanu początkowego.

Rozmiar populacji



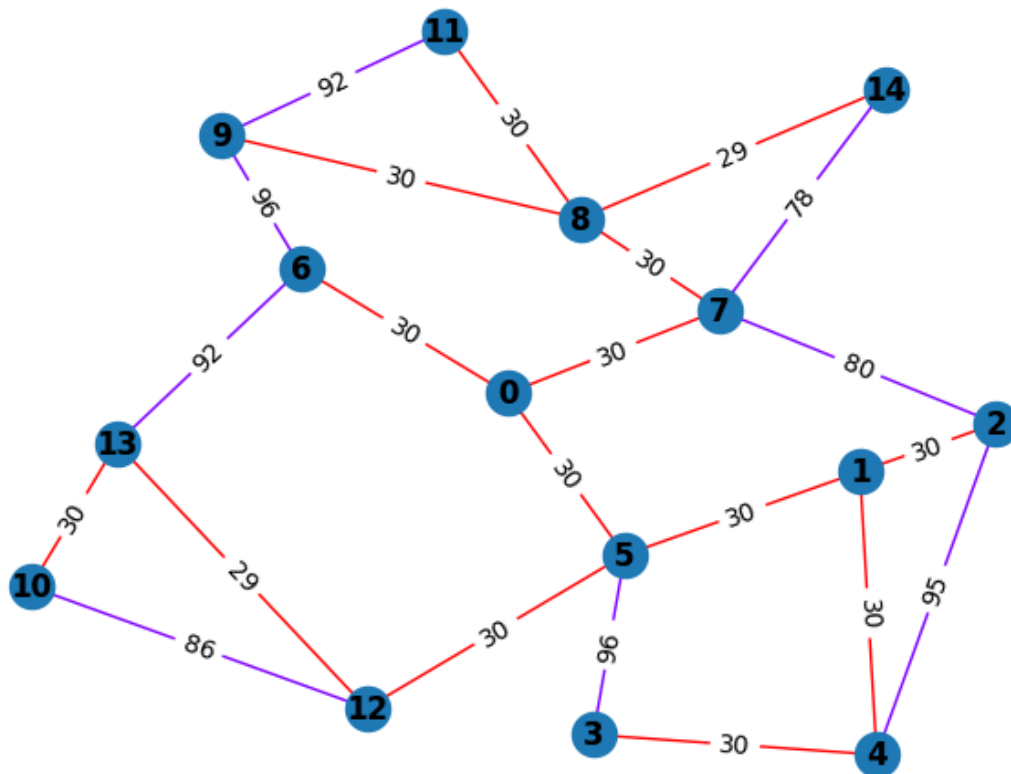
Obserwacje i wnioski:

- Pod względem samych iteracji, tym razem zdecydowanie najlepsze wyniki uzyskała populacja 2000 osobników: najszybszą zbieżność, najmniejsze odchylenia, najlepsze znalezione optimum.

- Natomiast porównując ewaluacje, najlepsze rozwiązanie znalazło 500 osobników, zaś najlepszą i najszybciej zbiegającą średnią miało 200 osobników.

Sieć rzeczywista “atlanta”

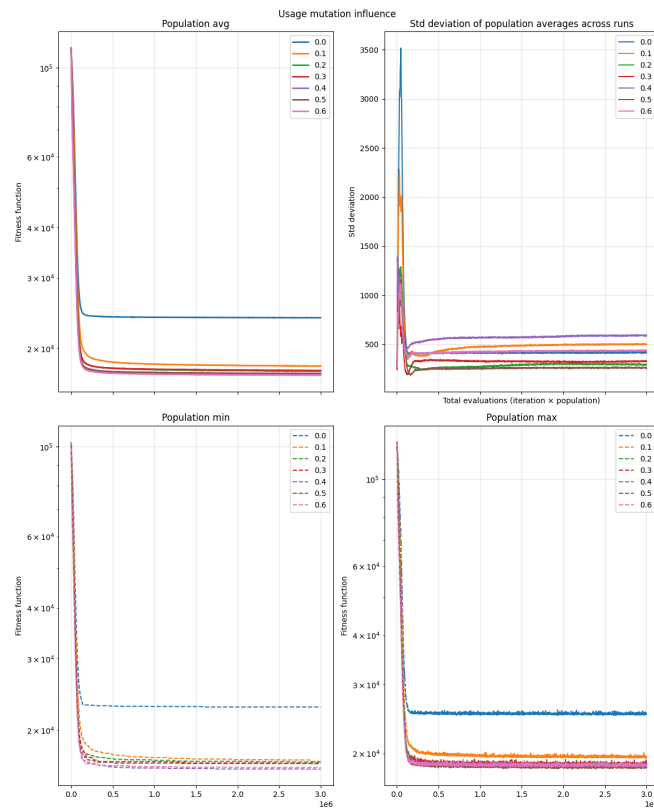
Wynik modelu MIP



Łączny koszt: 3780.

Algorytm ewolucyjny

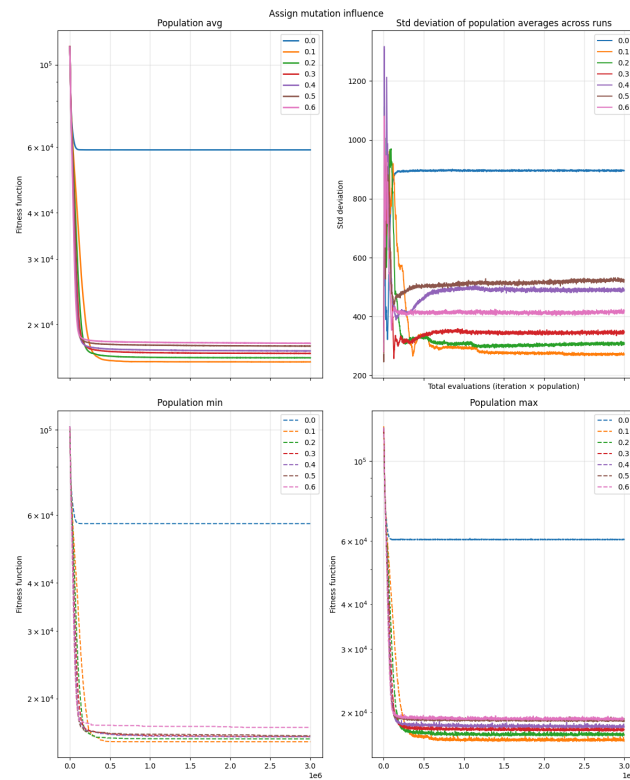
Mutacja zajętości krawędzi



Obserwacje i wnioski:

- Wartość 0.1 uzyskała najgorszą zbieżność i średnią populacji (spośród niezerowych wartości).
- Wartości 0.4 i 0.6 uzyskały najlepsze wyniki najlepszych osobników, ale 0.4 miała również najwyższe odchylenia.
- Najniższe odchylenia nastąpiły dla wartości 0.5.

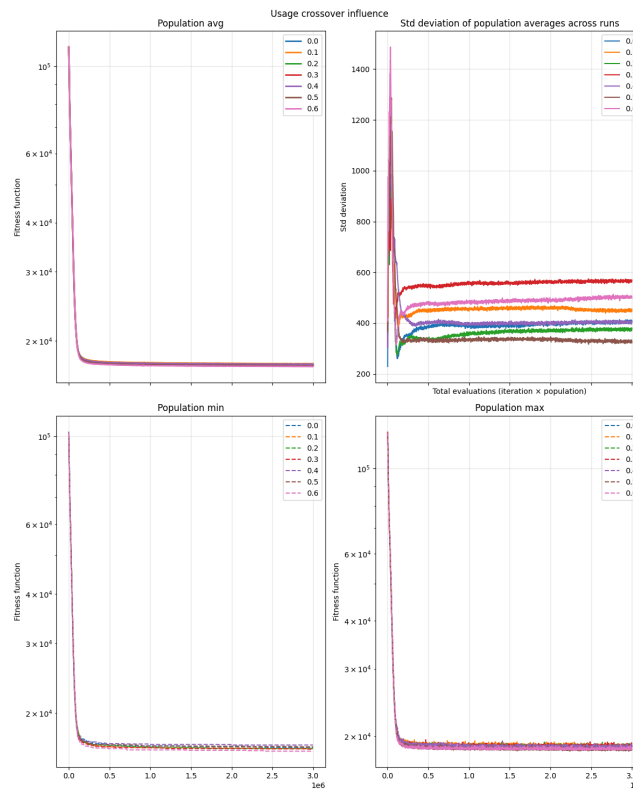
Mutacja wyboru transpondera



Obserwacje i wnioski:

- Podobnie jak dla sieci full-mesh 10 węzłów, im mniejsza (*niezerowa*) szansa mutacji, tym lepsze są znalezione optima lokalne, lecz ich znalezienie dłużej zajmuje.

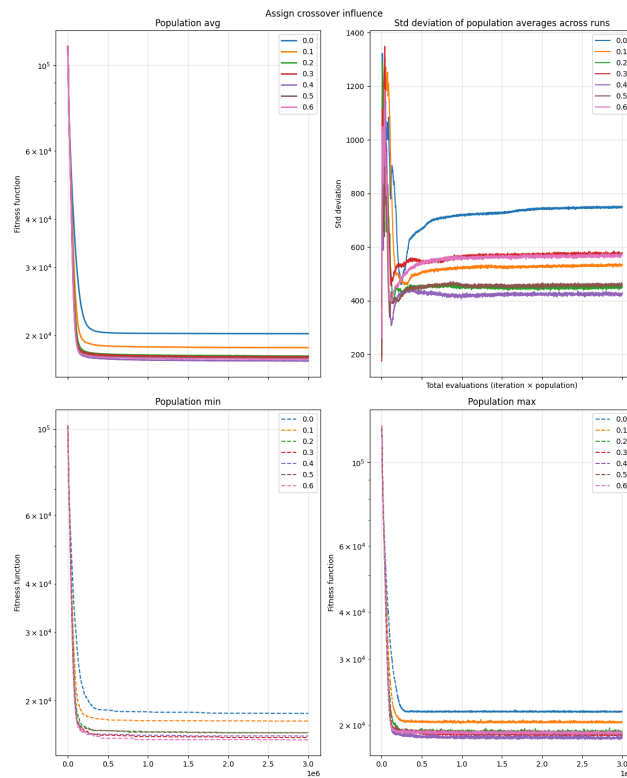
Krzyżowanie zajętości krawędzi



Obserwacje i wnioski:

- Dla sieci rzeczywistych ten parametr wydaje się nie mieć dużego znaczenia.
- Zauważalne są wyłącznie różnice w odchyleniach standardowych, lecz nie wydają się być jakkolwiek proporcjonalne do wybranych wartości. Największe odchylenia osiągnęła wartość 0.3, zaś najmniejsze — 0.5.

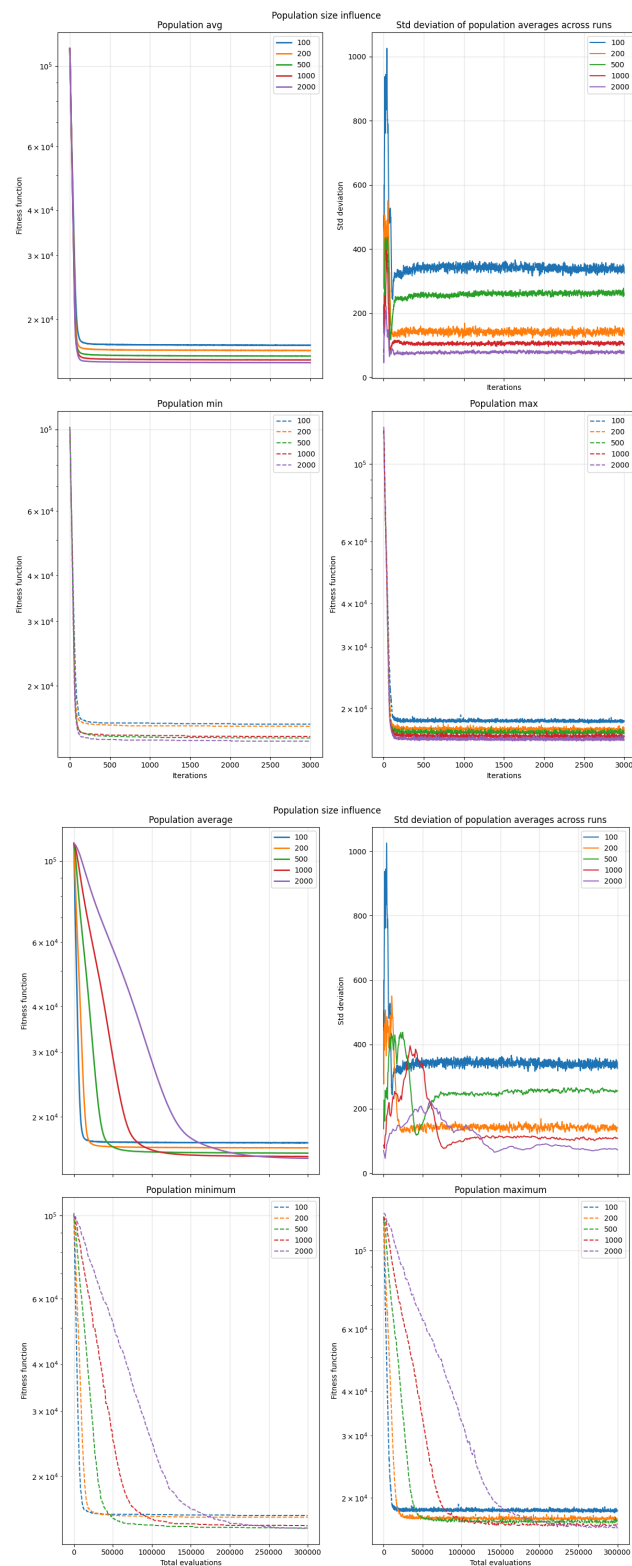
Krzyżowanie wyboru transpondera



Obserwacje i wnioski:

- Z niezerowych wartości parametru wyróżnia się tylko 0.1 jako najgorsza.
- Wartości 0.3 i 0.4 osiągnęły bardzo podobne wyniki zarówno pod względem krzywej zbieżności, jak i odchyłeń standardowych.

Rozmiar populacji



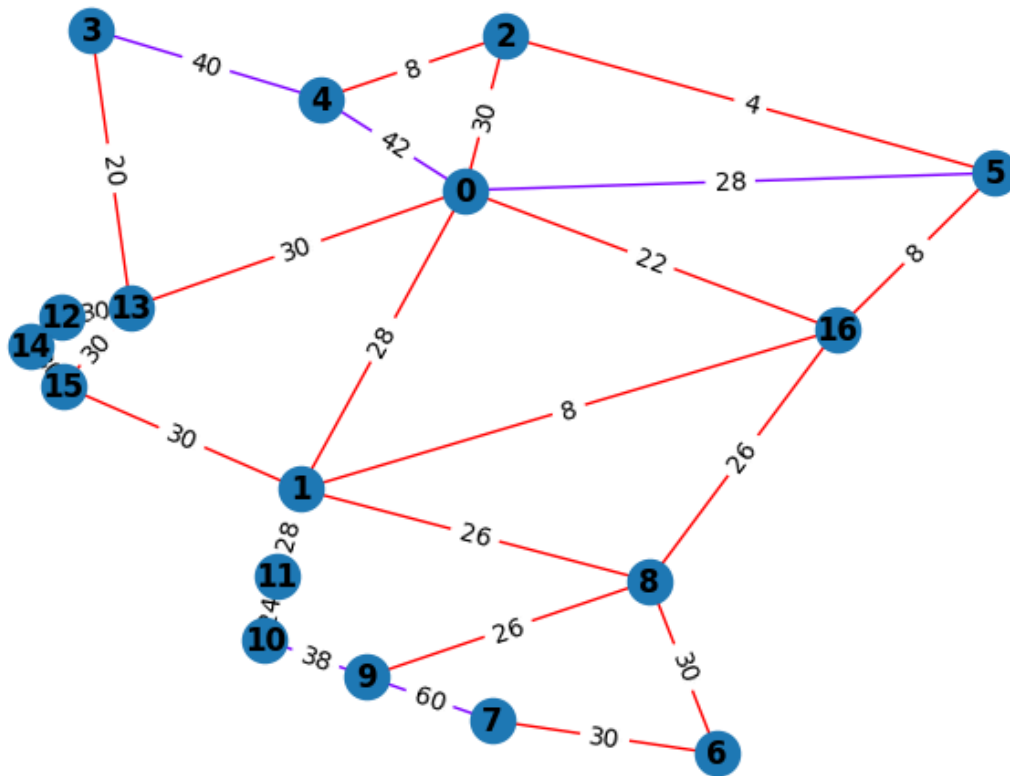
Obserwacje i wnioski:

- Pod względem samej liczby iteracji, 2000 osobników to zdecydowanie najlepsza opcja.
- Nawet przy porównaniu liczby ewaluacji, 2000 osobników mimo łagodnej krzywej zbieżności uzyskało najniższą średnią i najlepsze rozwiązanie.

- 500 osobników w porównaniu z 1000 osobników znalazło lepsze optimum, lecz miało gorszą średnią populacji (*i większe odchylenia*).

Sieć rzeczywista “nobel-germany”

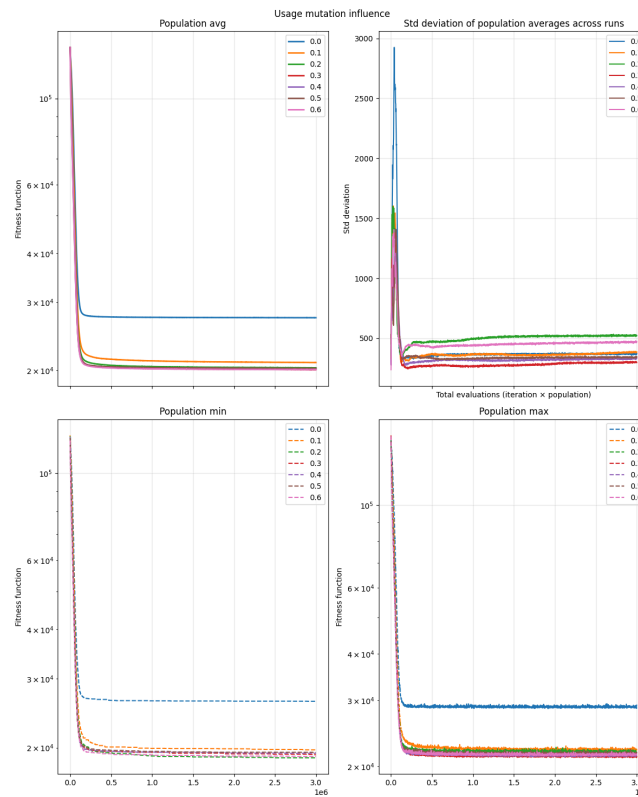
Wynik modelu MIP



Łączny koszt: 2176.

Algorytm ewolucyjny:

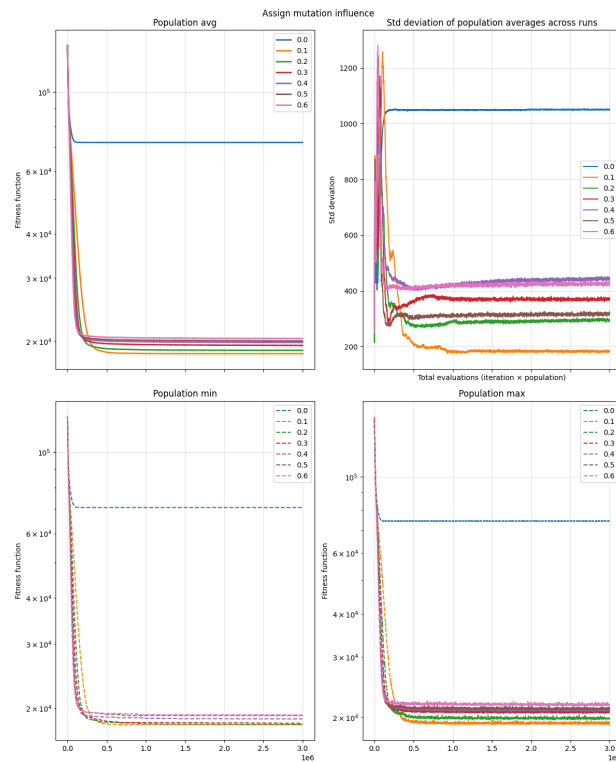
Mutacja zajętości krawędzi



Obserwacje i wnioski:

- Wartość 0.2 znalazła najlepsze optimum, lecz ma gorszą średnią populacji od podobnych wyników wartości 0.6.
- 0.1 się wyróżnia jako najgorsze z niezerowych wartości.

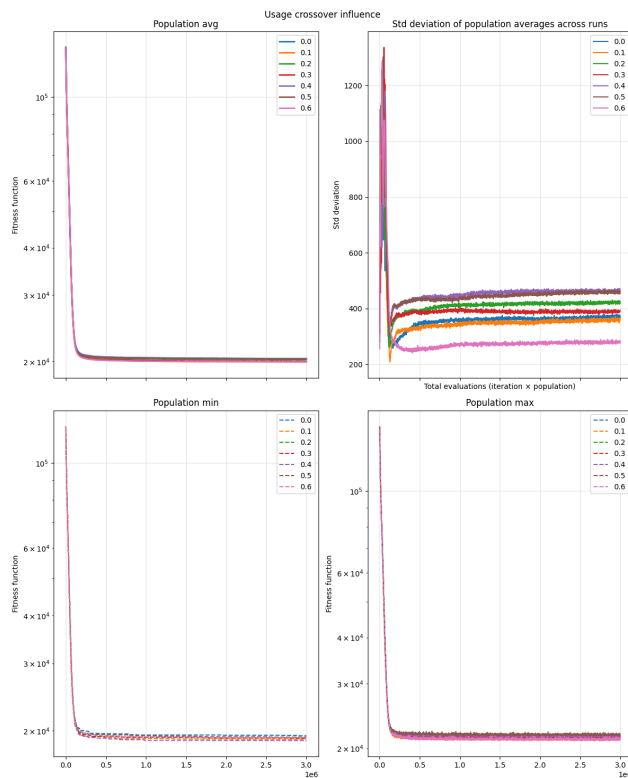
Mutacja wyboru transponderów:



Obserwacje i wnioski:

- Podobnie jak w poprzednich eksperymentach, im mniejsza wartość tego parametru, tym lepsze znalezione optimum, kosztem szybkości zbieżności.

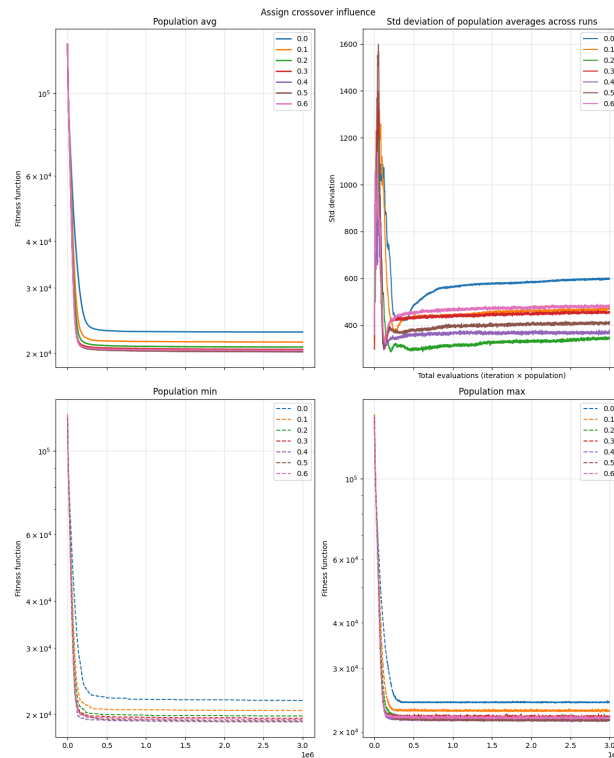
Krzyżowanie zajętości krawędzi:



Obserwacje i wnioski:

- Tu również dochodzimy do wniosku o braku znaczenia tego parametru dla sieci z małą liczbą krawędzi.

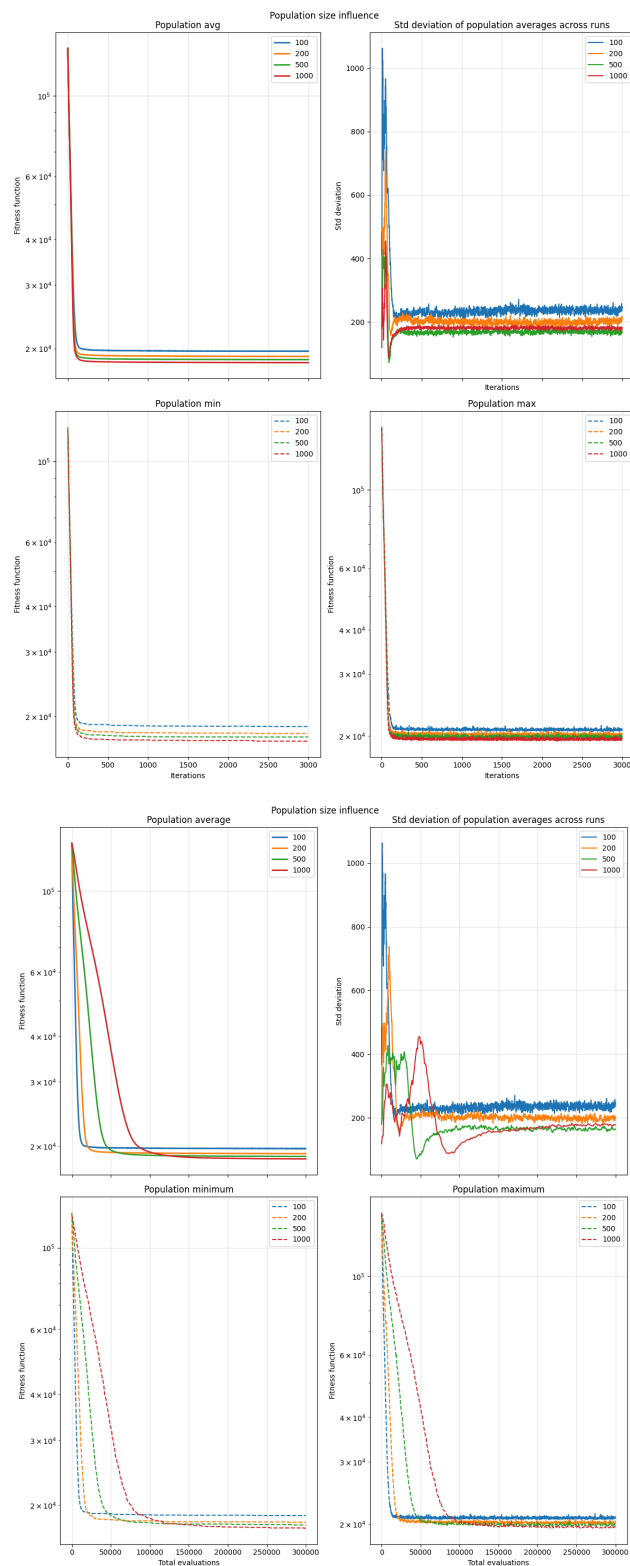
Krzyżowanie wyboru transpondera:



Obserwacje i wnioski:

- 0.1 jest tu najgorsze, pozostałe niezerowe wartości dają prawie identyczne wyniki.

Rozmiar populacji

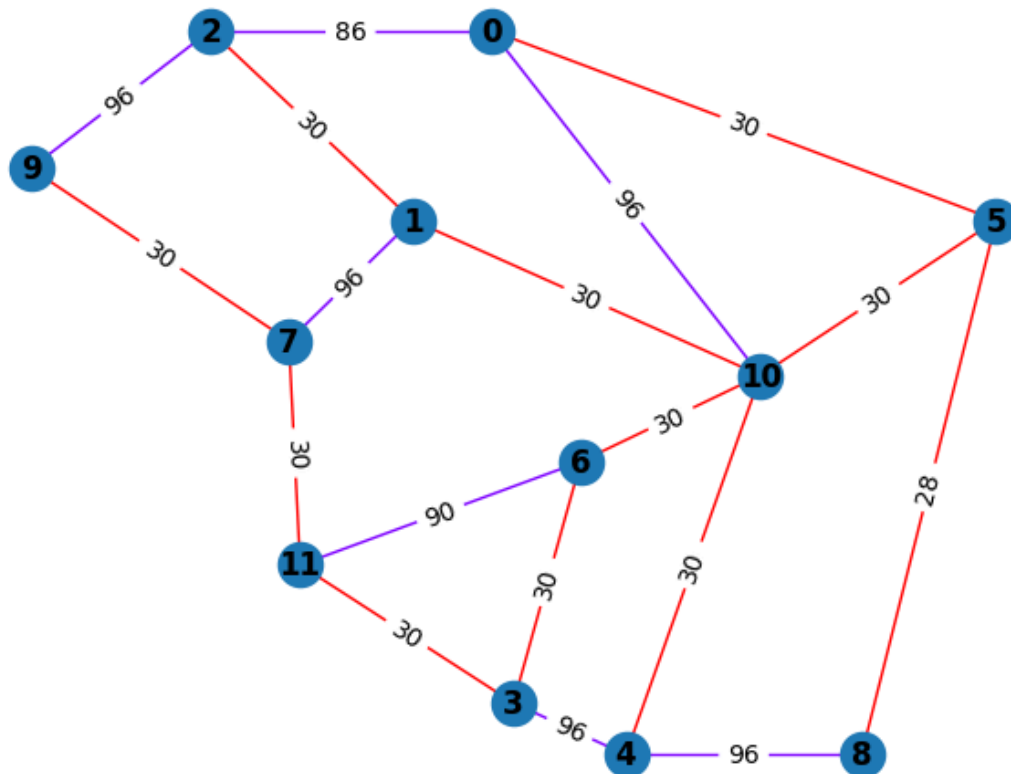


Obserwacje i wnioski:

- Najlepsze wyniki uzyskało 1000 osobników, choć pod względem łącznych ewaluacji krzywa zbieżności wyszła mniej stroma.

Sieć rzeczywista "Polska"

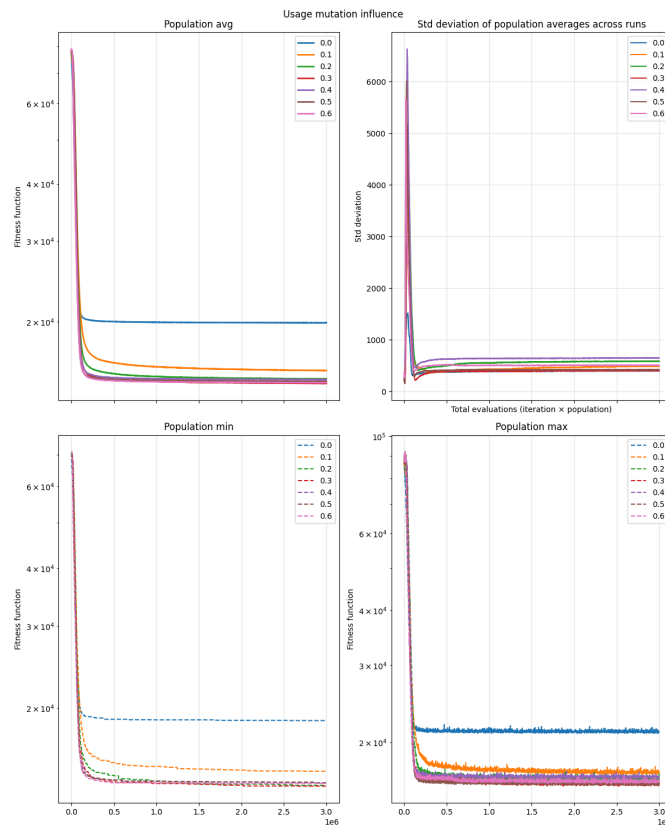
Wynik modelu MIP



Łączny koszt: 4356.

Algorytm ewolucyjny:

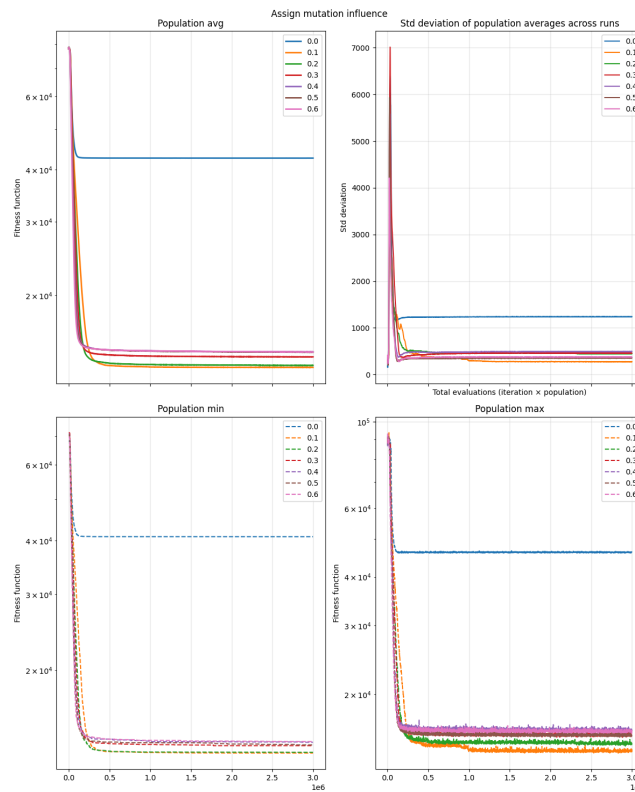
Mutacja zajętości krawędzi



Obserwacje i wnioski:

- Podobne wyniki co pozostałe sieci rzeczywiste — krzywe prawie identyczne, z wyjątkiem wartości 0.1, która pozostaje najgorsza.

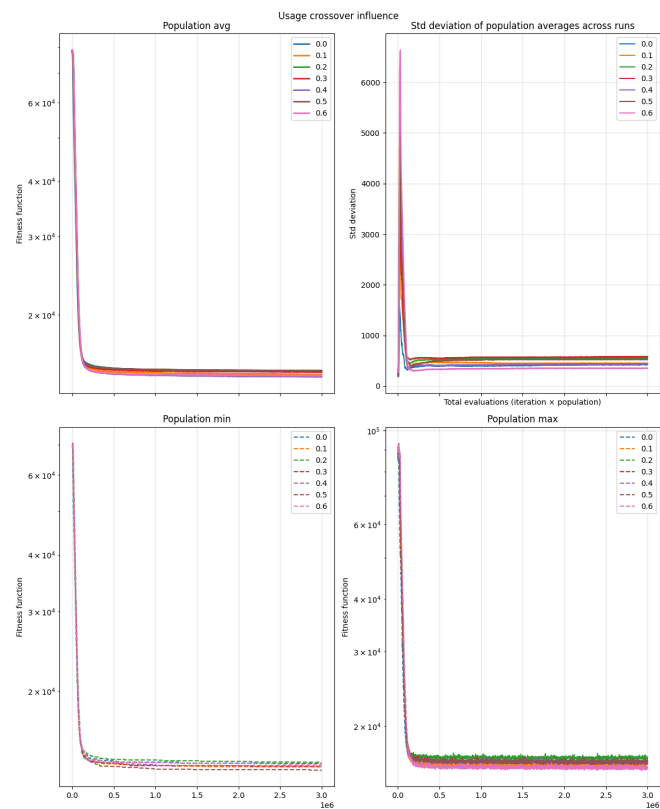
Mutacja wyboru transponderów:



Obserwacje i wnioski:

- Tu również zauważalne jest, że małe, niezerowe wartości parametru zapewniają najlepsze wyniki, ale z nieco wolniejszą zbieżnością.

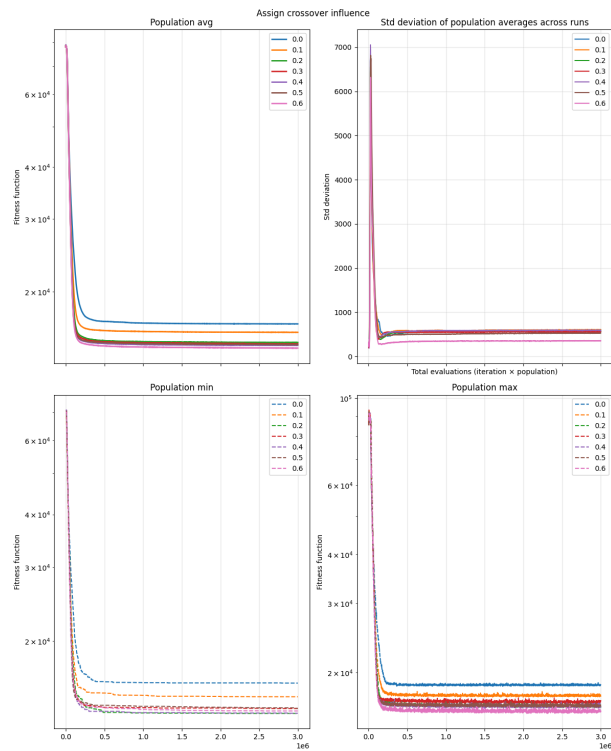
Krzyżowanie zajętości krawędzi:



Obserwacje i wnioski:

- Brak zauważalnego znaczenia parametru.

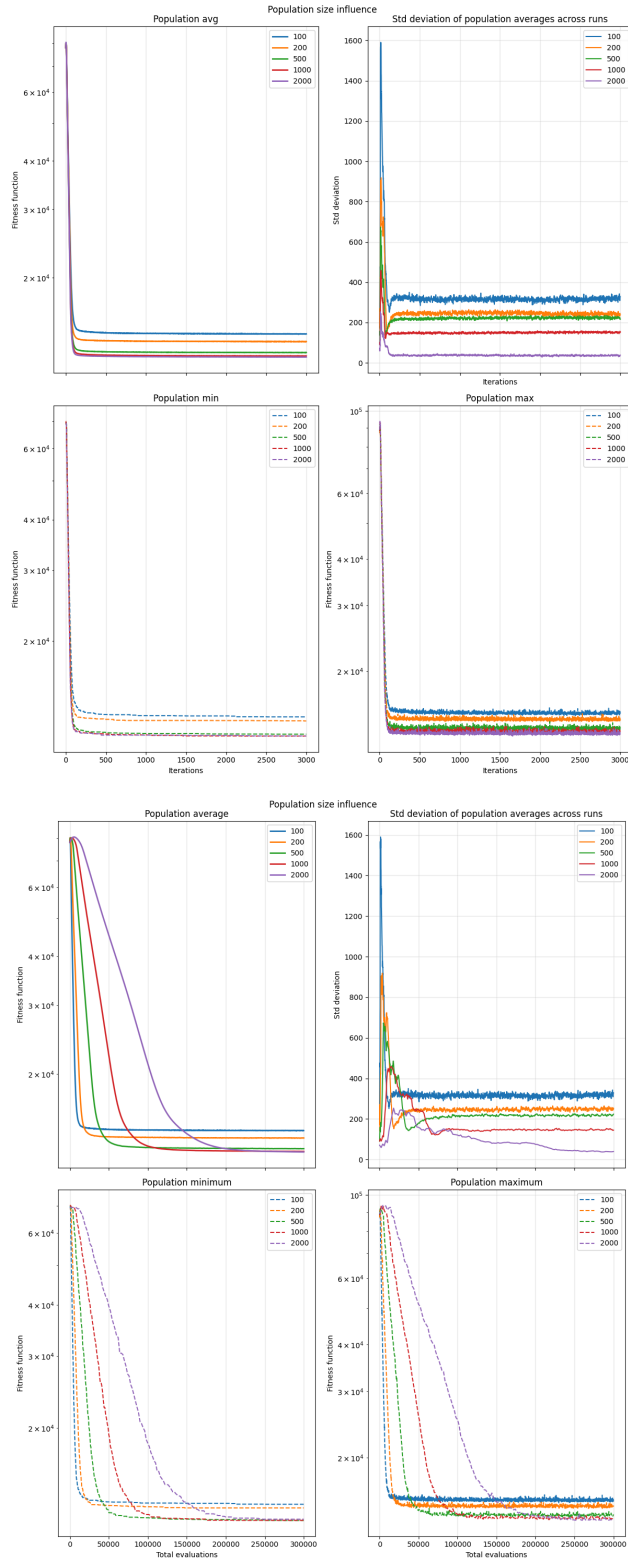
Krzyżowanie wyboru transpondera:



Obserwacje i wnioski:

- Podobne wyniki co do poprzednich eksperymentów, czyli znacząco wyróżnia się tylko wartość 0.1 jako najgorsza.
- Wartość 0.6 uzyskała najlepszą średnią populacji (*i co jest z tym poniekąd powiązane, najmniejsze odchylenia standardowe*).

Rozmiar populacji



Obserwacje i wnioski:

- Najlepsze wyniki, w szczególności najmniejsze odchylenia standardowe, uzyskała populacja 2000 osobników.
- Podobnie, jak w poprzednich eksperymentach, im większa populacja, tym wolniejsza zbieżność w skali łącznej liczby ewaluacji.

Podsumowanie wniosków

Wartości parametrów

- Szansa na mutację zajętości krawędzi ogólnie powinna być coraz mniejsza wraz z rosnącą liczbą krawędzi ze względu na ryzyko błędzenia w przestrzeni niedopuszczalnych rozwiązań.
- Aby znaleźć jak najlepsze rozwiązanie, szansa na mutację wyboru transponderów powinna być jak najmniejsza (*ale nie zerowa*), z uwagą na rosnącą liczbę potrzebnych ewaluacji do znalezienia optimum.
- Parametr CR krzyżowania zajętości krawędzi (*czyli "jednorodność" genów potomków*) nie ma większego znaczenia dla sieci z małą liczbą krawędzi.
- Parametr CR krzyżowania wyboru transponderów powinien być większy od 0.1.
- Ogólnie im większa populacja, tym lepiej, lecz w przypadku akceleracji sprzętowej poprzez GPU zdecydowanie należy mieć na uwadze swoje zasoby sprzętowe.
- Małe populacje mają zaletę bardzo szybkiej zbieżności, nawet jeśli nie są w stanie znaleźć równie dobrego rozwiązania, co większe populacje w tej samej liczbie ewaluacji osobników.
- W przypadku sieci o dużej liczbie krawędzi (*np. full-mesh 10 węzłów*) dochodzi do przełamania reguły o szybszej zbieżności małych populacji (*ale tylko dla średniego wyniku populacji — najlepsze osobniki dalej szybciej dążą do lokalnego optimum*). Takie odchylenia są zapewne spowodowane przez znacznie bardziej skomplikowaną strukturę sieci.

Pozostałe wnioski

Patrząc na sam czas wykonywania komórek kodu (*wyświetlany w Jupyterze po uruchomieniu*), mimo stosowanych wielu usprawnień, algorytm ewolucyjny nie wydaje się być efektywniejszy do tego rodzaju zadania niż dokładny model MIP, głównie ze względu na swoją inherentną losowość i błędzenie po wielowymiarowej przestrzeni przeszukiwań.

Możliwymi kierunkami rozwoju algorytmu mogłyby być dalsze rozbięcie zadania na podproblemy (*np. osobne optymalizowanie liczby ścieżek i wyboru transponderów*) bądź zastosowanie więcej technik, które poznaliśmy na końcowych wykładach przedmiotu POP (*naprawianie osobników; hybrydyzacja z lokalną optymalizacją — np. wspomaganie mutacji poprzez wyznaczanie od razu całych ścieżek zamiast ich fragmentów*).

Należy mieć na uwadze, że bardziej rozwinięte sposoby mutacji lub reprezentacji osobników mogą utrudnić sformułowanie sensownego krzyżowania, lecz nasze eksperymenty i tak wykazały, że krzyżowanie dla tego problemu nie ma aż tak dużego znaczenia, jak mutacja.

Opis wykorzystanych narzędzi, bibliotek, itp.

Najważniejsze z wykorzystanych bibliotek pythonowych to:

- **amplpy** do integracji AMPL z Pythonem;
- **networkx** do wizualizacji i niektórych operacji na grafach;
- **torch** do akceleracji sprzętowej obliczeń poprzez jednostki CUDA;
- **matplotlib** do wizualizacji danych wejściowych i osobników;

Dodatkowo użyliśmy menedżera pakietów **uv** aby zagwarantować zgodność tych zależności.